

59)

1. Insert After Every Even Node Problem Statement: Given a singly linked list, insert a new node with value X immediately after every node containing an even number. Input N Linked List Elements X
Output Modified Linked List .Sample Input 5 2 5 8 7 4 10 Sample Output 2 10 5 8 10 7 4 10

```
#include <stdio.h>
```

```
#include<stdlib.h>
```

```
struct node{
```

```
    int data;
```

```
    struct node *next;
```

```
}*head=NULL;
```

```
void append(int num){
```

```
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
```

```
    newnode->data=num;
```

```
    newnode->next=NULL;
```

```
    if(head==NULL){
```

```
        head=newnode;
```

```
    }
```

```
    else{
```

```
        struct node *temp=head;
```

```
        while(temp->next!=NULL){
```

```
            temp=temp->next;
```

```
        }
```

```
        temp->next=newnode;
```

```
    }
```

```
}
```

```
void insatevenum(int n,int a){
```

```
    struct node *temp=head;
```

```
    for(int i=0;i<n;i++){
```

```

        if((temp->data)%2==0){
            struct node *newnode=(struct node*)malloc(sizeof(struct node));
            newnode->data=a;
            newnode->next=temp->next;
            temp->next=newnode;
            temp=temp->next;
        }
        temp=temp->next;
    }

}

void display(){
    struct node *temp=head;
    while(temp->next!=NULL){
        printf("%d\n",temp->data);
        temp=temp->next;
    }
    printf("%d",temp->data);
}

int main()
{
    int n,num,a;
    scanf("%d",&n);
    for(int i=0;i<n;i++){
        scanf("%d",&num);
        append(num);
    }
    printf("enter value:");
    scanf("%d",&a);
    insatevenum(n,a);
    display();
}

```

```
    return 0;
}
```

60)

2. Delete Every K-th Node Problem Statement: Delete every K-th node from the linked list until the end. Input N Elements K Output Updated Linked List Sample Input 8 1 2 3 4 5 6 7 8 3 Sample Output 1 2 4 5 7 8

```
#include <stdio.h>
```

```
#include<stdlib.h>
```

```
struct node{
    int data;
    struct node *next;
}*head=NULL;
```

```
void append(int num){
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
    newnode->data=num;
    newnode->next=NULL;
    if(head==NULL){
        head=newnode;
    }
    else{
        struct node *temp=head;
        while(temp->next!=NULL){
            temp=temp->next;
        }
        temp->next=newnode;
    }
}
```

```

void delknode(int a){

    struct node *temp=head;
    for(int i=0;i<a-2;i++){
        temp=temp->next;
    }
    struct node *del=temp->next;
    temp->next=temp->next->next;
    free(del);

}

void display(){
    struct node *temp=head;
    while(temp->next!=NULL){
        printf("%d\n",temp->data);
        temp=temp->next;
    }
    printf("%d",temp->data);
}

int main()
{
    int n,num,a;
    scanf("%d",&n);
    for(int i=0;i<n;i++){
        scanf("%d",&num);
        append(num);
    }
    printf("enter node:");
    scanf("%d",&a);
    delknode(a);
    display();
}

```

```
    return 0;
}
```

61)Reverse a linked list:

```
#include <stdio.h>
```

```
#include<stdlib.h>
```

```
struct node{
    int data;
    struct node *next;
}*head=NULL;
```

```
void append(int num){
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
    newnode->data=num;
    newnode->next=NULL;
    if(head==NULL){
        head=newnode;
    }
    else{
        struct node *temp=head;
        while(temp->next!=NULL){
            temp=temp->next;
        }
        temp->next=newnode;
    }
}
```

```
void reverse(){
    struct node *temp=head;
```

```

    struct node *prev=NULL;
    struct node *next=NULL;
    while(temp!=NULL){
        next=temp->next;
        temp->next=prev;
        prev=temp;
        temp=next;
    }
    head=prev;

}

void display(){
    struct node *temp=head;
    while(temp->next!=NULL){
        printf("%d\n",temp->data);
        temp=temp->next;
    }
    printf("%d",temp->data);
}

int main()
{
    int n,num,a;
    scanf("%d",&n);
    for(int i=0;i<n;i++){
        scanf("%d",&num);
        append(num);
    }

    reverse();
    display();
}

```

```
    return 0;
}
```

62)

4. Rotate the Linked List by K Positions Problem Statement: Rotate the linked list to the left by K positions. Input N Elements K Output Rotated Linked List Sample Input 6 10 20 30 40 50 60 2 Sample Output 30 40 50 60 10 20

```
#include <stdio.h>
```

```
#include<stdlib.h>
```

```
struct node{
    int data;
    struct node *next;
}*head=NULL;
```

```
void append(int num){
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
    newnode->data=num;
    newnode->next=NULL;
    if(head==NULL){
        head=newnode;
    }
    else{
        struct node *temp=head;
        while(temp->next!=NULL){
            temp=temp->next;
        }
        temp->next=newnode;
    }
}
```

```
void rotate(int a){
    struct node *temp=head;
```

```

    struct node *temp1=head;
    for(int i=0;i<a-1;i++){
        temp=temp->next;
    }
    struct node *dhead=temp->next;
    while(temp1->next){
        temp1=temp1->next;
    }
    temp1->next=head;
    head=temp->next;
    temp->next=NULL;
}

void display(){
    struct node *temp=head;
    while(temp->next!=NULL){
        printf("%d\n",temp->data);
        temp=temp->next;
    }
    printf("%d",temp->data);
}

int main()
{
    int n,num,a;
    scanf("%d",&n);
    for(int i=0;i<n;i++){
        scanf("%d",&num);
        append(num);
    }
    printf("enter k:");
    scanf("%d",&a);
    rotate(a);
}

```



```
display();
```

```
return 0;
```

```
}
```

63)

5. Remove Consecutive Duplicate Nodes Problem Statement: Given a sorted linked list, remove all consecutive duplicate nodes. Input N Sorted Elements Output Linked List without duplicates Sample Input 9 1 1 2 2 2 4 4 5 5 Sample Output 1 2 4 5

```
#include <stdio.h>
```

```
#include<stdlib.h>
```

```
struct node{
```

```
    int data;
```

```
    struct node *next;
```

```
}*head=NULL;
```

```
void append(int num){
```

```
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
```

```
    newnode->data=num;
```

```
    newnode->next=NULL;
```

```
    if(head==NULL){
```

```
        head=newnode;
```

```
    }
```

```
    else{
```

```
        struct node *temp=head;
```

```
        while(temp->next!=NULL){
```

```
            temp=temp->next;
```

```
        }
```

```
        temp->next=newnode;
```

```
    }
```

```
}
```

```

void remove1(){
    struct node *temp1=head;
    struct node *temp2=head;
    while(temp1!=NULL){
        struct node *prev=temp1;
        temp2=temp1->next;
        while(temp2!=NULL){
            if(temp1->data == temp2->data){
                struct node *del=temp2;
                prev->next=temp2->next;
                temp2=prev->next;
                free(del);
            }
            else{
                prev=temp2;
                temp2=temp2->next;
            }
        }
        temp1=temp1->next;
    }
}

void display(){
    struct node *temp=head;
    while(temp->next!=NULL){
        printf("%d\n",temp->data);
        temp=temp->next;
    }
    printf("%d",temp->data);
}

int main()
{

```

```

int n,num,a;

scanf("%d",&n);

for(int i=0;i<n;i++){

    scanf("%d",&num);

    append(num);

}

remove1();

display();

return 0;

}

```

64)

10. Replace Every Node with the Sum of Itself and Next Node Problem Statement: Replace each node's value with the sum of itself and the next node. The last node remains unchanged. Input N Elements Output Modified Linked List Sample Input 5 2 4 6 8 10 Sample Output 6 10 14 18 10

```
#include <stdio.h>
```

```
#include<stdlib.h>
```

```

struct node{

    int data;

    struct node *next;

}*head=NULL;

```

```

void append(int num){

    struct node *newnode=(struct node*)malloc(sizeof(struct node));

    newnode->data=num;

    newnode->next=NULL;

    if(head==NULL){

        head=newnode;

    }

    else{

```

```

    struct node *temp=head;
    while(temp->next!=NULL){
        temp=temp->next;
    }
    temp->next=newnode;
}
}

```

```

void add(){
    struct node *temp=head;
    while(temp->next){
        temp->data=temp->data + temp->next->data;
        temp=temp->next;
    }
}

```

```

void display(){
    struct node *temp=head;
    while(temp->next!=NULL){
        printf("%d\n",temp->data);
        temp=temp->next;
    }
    printf("%d",temp->data);
}

```

```

int main()
{
    int n,num;
    scanf("%d",&n);
    for(int i=0;i<n;i++){
        scanf("%d",&num);
        append(num);
    }
}

```

```

    add();

    display();

    return 0;
}

```

65)

9. Split Linked List into Even and Odd Lists Problem Statement: Split the given linked list into two separate linked lists: ● One containing even-valued nodes. ● One containing odd-valued nodes. Maintain the original order in both lists. Input N Elements Output Even List Odd List Sample Input 8 2 7 8 5 6 9 4 1 Sample Output Even List: 2 8 6 4 Odd List: 7 5 9 1

```
#include <stdio.h>
```

```
#include<stdlib.h>
```

```

struct node{
    int data;

    struct node *next;
}*head=NULL;

struct node1{
    int data;

    struct node1 *next;
}*head1=NULL;

struct node2{
    int data;

    struct node2 *next;
}*head2=NULL;

```

```

void append(int num){
    struct node *newnode=(struct node*)malloc(sizeof(struct node));

    newnode->data=num;

    newnode->next=NULL;

    if(head==NULL){

```

```

        head=newnode;
    }
    else{
        struct node *temp=head;
        while(temp->next!=NULL){
            temp=temp->next;
        }
        temp->next=newnode;
    }
}

void append1(int num){
    struct node1 *newnode=(struct node1*)malloc(sizeof(struct node1));
    newnode->data=num;
    newnode->next=NULL;
    if(head1==NULL){
        head1=newnode;
    }
    else{
        struct node1 *temp=head1;
        while(temp->next!=NULL){
            temp=temp->next;
        }
        temp->next=newnode;
    }
}

void append2(int num){
    struct node2 *newnode=(struct node2*)malloc(sizeof(struct node2));
    newnode->data=num;
    newnode->next=NULL;
    if(head2==NULL){
        head2=newnode;
    }
}

```

```

    }
    else{
        struct node2 *temp=head2;
        while(temp->next!=NULL){
            temp=temp->next;
        }
        temp->next=newnode;
    }
}

```

```

void split(){
    struct node *temp=head;
    while(temp!=NULL){
        if((temp->data)%2==0){
            append1(temp->data);
        }else{
            append2(temp->data);
        }
        temp=temp->next;
    }
}

```

```

void display1(){
    printf("Even list:\n");
    struct node1 *temp=head1;
    while(temp!=NULL){
        printf("%d\n",temp->data);
        temp=temp->next;
    }
}

```

```

void display2(){
    printf("Odd list\n");
}

```

```

    struct node2 *temp=head2;
    while(temp!=NULL){
        printf("%d\n",temp->data);
        temp=temp->next;
    }
}

```

```

int main()
{
    int n,num;
    scanf("%d",&n);
    for(int i=0;i<n;i++){
        scanf("%d",&num);
        append(num);
    }
}

```

```

    split();
    display1();
    display2();
    return 0;
}

```

66)Sort linked list:

```

#include <stdio.h>
#include<stdlib.h>

```

```

struct node{
    int data;
    struct node *next;
}*head=NULL;

```

```

void append(int num){
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
}

```



```

newnode->data=num;
newnode->next=NULL;
if(head==NULL){
    head=newnode;
}
else{
    struct node *temp=head;
    while(temp->next!=NULL){
        temp=temp->next;
    }
    temp->next=newnode;
}
}

void sort(){
    struct node *temp1=head;
    while(temp1->next!=NULL){
        struct node *temp2=head;
        while(temp2->next!=NULL){
            if(temp2->data > temp2->next->data){
                int t=temp2->data;
                temp2->data=temp2->next->data;
                temp2->next->data=t;
            }
            temp2=temp2->next;
        }
        temp1=temp1->next;
    }
}

void display(){

```

```

    struct node *temp=head;
    while(temp->next!=NULL){
        printf("%d\n",temp->data);
        temp=temp->next;
    }
    printf("%d",temp->data);
}

int main()
{
    int n,num;
    scanf("%d",&n);
    for(int i=0;i<n;i++){
        scanf("%d",&num);
        append(num);
    }

    sort();
    display();

    return 0;
}

```

67) Question 1: Display all nodes in forward order:

```
#include <stdio.h>
```

```
#include<stdlib.h>
```

```

struct node{
    int data;
    struct node *next;
    struct node *prev;
}*head=NULL;

```

```

void append(int num){
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
    newnode->prev=NULL;
    newnode->data=num;
    newnode->next=NULL;
    if(head==NULL){
        head=newnode;
    }
    else{
        struct node *temp=head;
        while(temp->next!=NULL){
            temp=temp->next;
        }
        temp->next=newnode;
        newnode->prev=temp;
    }
}

```

```

void display(){
    struct node *temp=head;
    while(temp->next!=NULL){
        printf("%d\n",temp->data);
        temp=temp->next;
    }
    printf("%d",temp->data);
}

```

```

int main()
{
    int n,num;
    scanf("%d",&n);

```

```

for(int i=0;i<n;i++){
    scanf("%d",&num);
    append(num);
}

display();

return 0;
}

```

68) Question 2: Display all nodes in reverse order:

```
#include <stdio.h>
```

```
#include<stdlib.h>
```

```

struct node{
    int data;
    struct node *next;
    struct node *prev;
}*head=NULL;

```

```

void append(int num){
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
    newnode->prev=NULL;
    newnode->data=num;
    newnode->next=NULL;
    if(head==NULL){
        head=newnode;
    }
    else{
        struct node *temp=head;
        while(temp->next!=NULL){

```

```

        temp=temp->next;
    }
    temp->next=newnode;
    newnode->prev=temp;
}
}

```

```

void display(){
    struct node *temp=head;
    while(temp->next!=NULL){
        temp=temp->next;
    }
    while(temp!=head){
        printf("%d ",temp->data);
        temp=temp->prev;
    }
    printf("%d ",temp->data);
}

```

```

int main()
{
    int n,num;
    scanf("%d",&n);
    for(int i=0;i<n;i++){
        scanf("%d",&num);
        append(num);
    }

```

```

display();

```

```

return 0;

```

```
}
```

69) Question 3: Insert a node at the beginning:

```
#include <stdio.h>
```

```
#include<stdlib.h>
```

```
struct node{
```

```
    int data;
```

```
    struct node *next;
```

```
    struct node *prev;
```

```
}*head=NULL;
```

```
void append(int num){
```

```
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
```

```
    newnode->prev=NULL;
```

```
    newnode->data=num;
```

```
    newnode->next=NULL;
```

```
    if(head==NULL){
```

```
        head=newnode;
```

```
    }
```

```
    else{
```

```
        struct node *temp=head;
```

```
        while(temp->next!=NULL){
```

```
            temp=temp->next;
```

```
        }
```

```
        temp->next=newnode;
```

```
        newnode->prev=temp;
```

```
    }
```

```
}
```

```
void insatbeg(int a){
```

```
    struct node *temp=head;
```

```

    struct node *newnode=(struct node*)malloc(sizeof(struct node));
    newnode->prev=NULL;
    newnode->data=a;
    newnode->next=head;
    temp->prev=newnode;
    head=newnode;
}

```

```

void display(){
    struct node *temp=head;
    while(temp->next!=NULL){
        printf("%d ",temp->data);
        temp=temp->next;
    }
    printf("%d \n",temp->data);
}

```

```

int main()
{
    int n,num,a,pos;
    scanf("%d",&n);
    for(int i=0;i<n;i++){
        scanf("%d",&num);
        append(num);
    }

```

```

    printf("enter num:");
    scanf("%d",&a);
    display();
    insatbeg(a);
    display();

```

```
    return 0;
}
```

70) Question 4: Insert a node at the end:

```
#include <stdio.h>
```

```
#include<stdlib.h>
```

```
struct node{
    int data;
    struct node *next;
    struct node *prev;
}*head=NULL;
```

```
void append(int num){
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
    newnode->prev=NULL;
    newnode->data=num;
    newnode->next=NULL;
    if(head==NULL){
        head=newnode;
    }
    else{
        struct node *temp=head;
        while(temp->next!=NULL){
            temp=temp->next;
        }
        temp->next=newnode;
        newnode->prev=temp;
    }
}
```



```

void display(){
    struct node *temp=head;
    while(temp->next!=NULL){
        printf("%d ",temp->data);
        temp=temp->next;
    }
    printf("%d \n",temp->data);
}

int main()
{
    int n,num,a,pos;
    scanf("%d",&n);
    for(int i=0;i<n;i++){
        scanf("%d",&num);
        append(num);
    }

    printf("enter num:");
    scanf("%d",&a);
    display();
    append(a);
    display();

    return 0;
}

```

71) Question 5: Count the total number of nodes:

```
#include <stdio.h>
```

```
#include<stdlib.h>
```

```
struct node{
```

```

int data;

struct node *next;

struct node *prev;
}*head=NULL;

void append(int num){
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
    newnode->prev=NULL;
    newnode->data=num;
    newnode->next=NULL;
    if(head==NULL){
        head=newnode;
    }
    else{
        struct node *temp=head;
        while(temp->next!=NULL){
            temp=temp->next;
        }
        temp->next=newnode;
        newnode->prev=temp;
    }
}

```

```

void count(){
    int c=0;
    struct node *temp=head;
    while(temp!=NULL){
        c++;
        temp=temp->next;
    }
    printf("count is %d",c);
}

```

```
}
```

```
void display(){  
    struct node *temp=head;  
    while(temp->next!=NULL){  
        printf("%d ",temp->data);  
        temp=temp->next;  
    }  
    printf("%d \n",temp->data);  
}
```

```
int main()  
{  
    int n,num,a,pos;  
    scanf("%d",&n);  
    for(int i=0;i<n;i++){  
        scanf("%d",&num);  
        append(num);  
    }  
    display();  
    count();  
  
    return 0;  
}
```

72) Question 2: Delete All Nodes in a Value Range (Doubly Linked List):

```
#include <stdio.h>
```

```
#include<stdlib.h>
```

```
struct node{  
    int data;  
    struct node *next;  
    struct node *prev;
```

```
}*head=NULL;
```

```
void append(int num){
```

```
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
```

```
    newnode->prev=NULL;
```

```
    newnode->data=num;
```

```
    newnode->next=NULL;
```

```
    if(head==NULL){
```

```
        head=newnode;
```

```
    }
```

```
    else{
```

```
        struct node *temp=head;
```

```
        while(temp->next!=NULL){
```

```
            temp=temp->next;
```

```
        }
```

```
        temp->next=newnode;
```

```
        newnode->prev=temp;
```

```
    }
```

```
}
```

```
void rev(int k, int l){
```

```
    struct node *temp=head;
```

```
    int a=0;
```

```
    while(temp->next!=NULL){
```

```
        if(temp->data>=k && temp->data<=l){
```

```
            a++;
```

```
            struct node *del=temp;
```

```
            temp->prev->next=temp->next;
```

```
            temp->next->prev=temp->prev;
```

```
            temp=temp->prev->next;
```

```
            free(del);
```

```

    }
    if(a==0){
        temp=temp->next;
    }
    a=0;
}
}

```

```

void display(){
    struct node *temp=head;
    while(temp->next!=NULL){
        printf("%d ",temp->data);
        temp=temp->next;
    }
    printf("%d \n",temp->data);
}

```

```

int main()
{
    int n,num,k,l;
    scanf("%d",&n);
    for(int i=0;i<n;i++){
        scanf("%d",&num);
        append(num);
    }
    printf("enter k:");
    scanf("%d",&k);
    printf("enter l:");
    scanf("%d",&l);
    display();
    rev(k,l);
    display();
}

```

```
    return 0;
}
```

73) Question 3: Find Pair Sum from Both Ends (Doubly Linked List):

```
#include <stdio.h>
```

```
#include<stdlib.h>
```

```
struct node{
    int data;
    struct node *next;
    struct node *prev;
}*head=NULL;
```

```
void append(int num){
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
    newnode->prev=NULL;
    newnode->data=num;
    newnode->next=NULL;
    if(head==NULL){
        head=newnode;
    }
    else{
        struct node *temp=head;
        while(temp->next!=NULL){
            temp=temp->next;
        }
        temp->next=newnode;
        newnode->prev=temp;
    }
}
```

```

void pairsum(int k){
    struct node *temp1=head;
    while(temp1->next){
        struct node *temp2=temp1->next;
        while(temp2!=NULL){
            if((temp1->data + temp2->data) == k){
                printf("%d %d\n",temp1->data,temp2->data);
            }
            temp2=temp2->next;
        }
        temp1=temp1->next;
    }
}

```

```

void display(){
    struct node *temp=head;
    while(temp->next!=NULL){
        printf("%d ",temp->data);
        temp=temp->next;
    }
    printf("%d \n",temp->data);
}

```

```

int main()
{
    int n,num,k;
    scanf("%d",&n);
    for(int i=0;i<n;i++){
        scanf("%d",&num);
        append(num);
    }
    printf("enter k:");
}

```

```

scanf("%d",&k);

//display();

pairsum(k);

return 0;
}

```

74) Question 4: Rotate Doubly List to the Right (Doubly Linked List):

```
#include <stdio.h>
```

```
#include<stdlib.h>
```

```

struct node{
    int data;
    struct node *next;
    struct node *prev;
}*head=NULL;

```

```

void append(int num){
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
    newnode->prev=NULL;
    newnode->data=num;
    newnode->next=NULL;
    if(head==NULL){
        head=newnode;
    }
    else{
        struct node *temp=head;
        while(temp->next!=NULL){
            temp=temp->next;
        }
        temp->next=newnode;
        newnode->prev=temp;
    }
}

```



```
    }  
}
```

```
void rotate(int k){  
    struct node *temp1=head;  
    while(temp1->next){  
        temp1=temp1->next;  
    }  
    for(int i=0;i<k-1;i++){  
        temp1=temp1->prev;  
    }  
    struct node *temp2=head;  
    head=temp1;  
    temp1->prev->next=NULL;  
    temp1->prev=NULL;  
    while(temp1->next){  
        temp1=temp1->next;  
    }  
    temp1->next=temp2;  
    temp2->prev=temp1;  
}
```

```
void display(){  
    struct node *temp=head;  
    while(temp->next!=NULL){  
        printf("%d ",temp->data);  
        temp=temp->next;  
    }  
    printf("%d \n",temp->data);  
}
```

```
int main()
```

```

{
    int n,num,k;
    scanf("%d",&n);
    for(int i=0;i<n;i++){
        scanf("%d",&num);
        append(num);
    }
    printf("enter k:");
    scanf("%d",&k);
    display();
    rotate(k);
    display();
    return 0;
}

```

75) Question 5: Remove Duplicate Blocks from Sorted DLL (Doubly Linked List):

```
#include <stdio.h>
```

```
#include<stdlib.h>
```

```

struct node{
    int data;
    struct node *next;
    struct node *prev;
}*head=NULL;

```

```

void append(int num){
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
    newnode->prev=NULL;
    newnode->data=num;
    newnode->next=NULL;
    if(head==NULL){
        head=newnode;
    }
}

```

```

    }
    else{
        struct node *temp=head;
        while(temp->next!=NULL){
            temp=temp->next;
        }
        temp->next=newnode;
        newnode->prev=temp;
    }
}

void rdup(){
    struct node *temp1=head;
    while(temp1!=NULL){
        struct node *temp2=temp1->next;
        while(temp2!=NULL){
            struct node *del=NULL;
            if(temp1->data == temp2->data){
                del=temp2;
                temp2->prev->next=temp2->next;
                if(temp2->next!=NULL){
                    temp2->next->prev=temp2->prev;
                }
            }
            temp2=temp2->next;
            free(del);
        }
        temp1=temp1->next;
    }
}

```

```

void display(){
    struct node *temp=head;
    while(temp->next!=NULL){
        printf("%d ",temp->data);
        temp=temp->next;
    }
    printf("%d \n",temp->data);
}

int main()
{
    int n,num;
    scanf("%d",&n);
    for(int i=0;i<n;i++){
        scanf("%d",&num);
        append(num);
    }

    display();
    rdup();
    display();
    return 0;
}

```

76) Circular Linked List Question 6: Display all elements once:

```
#include <stdio.h>
```

```
#include<stdlib.h>
```

```

struct node{
    int data;
    struct node *next;
    //struct node *prev;
}*head=NULL;

```

```

void append(int num){
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
    // newnode->prev=NULL;
    newnode->data=num;
    if(head==NULL){
        head=newnode;
        newnode->next=head;
    }
    else{
        struct node *temp=head;
        do{
            temp=temp->next;
        }while(temp->next!=head);
        temp->next=newnode;
        newnode->next=head;
        //newnode->prev=temp;
    }
}

```

```

void display(){
    struct node *temp=head;
    do{
        printf("%d ",temp->data);
        temp=temp->next;
    }while(temp->next!=head);
    printf("%d \n",temp->data);
}

```

```

int main()
{
    int n,num;

```

```
scanf("%d",&n);  
for(int i=0;i<n;i++){  
    scanf("%d",&num);  
    append(num);  
}  
  
display();  
return 0;  
}
```